The Journal of Systems and Software 228 (2025) 112479

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

So much more than test cases – An industrial study on testing of software units and components

Torvald Mårtensson [a,b]

[a] Linköping University, Linköping, Sweden
[b] Saab AB, Linköping, Sweden

ARTICLE INFO

Editor: Dr Daniel Mendez

Keywords:
Industrial study
In practice
Software testing
Software component
Software unit
Test tools
Test methods

ABSTRACT

While software-developing organizations increasingly write test cases and continuously run them to find defects and ensure quality, the associated maintenance and execution costs increase over time. According to practitioners in the software industry, more test cases is not the solution, which calls for other methods and tools that can complement traditional test cases. Based on a cross-company workshop with 30 participants, three focus groups with 19 participants, and a validation workshop with 30 participants, this paper presents insights from practitioners at six large-scale companies, describing experiences from real industrial settings with regards to testing of software units and components. As every tool is also a cost for the organization, this paper presents a structured and holistic approach based on grounded theory, where methods and tools are evaluated and selected from four categories, representing test activities with different purposes: Optimize feedback loops, Improve test efficiency, Improve test suite effectiveness, and Technical debt management. The participants in the study also described how methods and tools used within their companies were often not well described in the company's ways-of-working, which caused frustration and confusion. Feedback from the continuous integration and delivery pipeline must be understandable (describing what the problem is), actionable (describing what the developer needs to do), and explained (describing and motivating why this is important). The findings from this study provide actionable strategies for how testing of units and components can be conducted in a more structured way, which could help companies cut costs, save time and identify problems earlier.

1. Introduction

1.1. Background

In previous work, we have investigated how software companies should design their continuous integration and delivery pipelines to include test activities that support different stakeholders in the organization (Mårtensson et al., 2019) and have reported challenges with continuous practices observed in multiple industry settings (Ståhl and Mårtensson 2021). Our previous studies report on how the companies we studied responded to integration failures late in the pipeline by inserting "more and more test cases in earlier checks, but it hasn't helped", with one company relating that "out-of-control automated testing" had failed to deliver on their expectations. According to practitioners, increasing the number of test cases is not the solution, as our software systems are increasing in both size and complexity. This not only causes frustration for the developers in the organization, but is also a significant cost-driver for companies in the software industry. While software-developing organizations increasingly write test cases and continuously run them to find defects and ensure quality, their associated costs tend to increase over time, often significantly (Dobslaw et al. 2023).

In particular, the number of test cases at the unit and component level in the pipeline keeps on growing, requiring ever increasing resources to execute until the straight-forward solution of investing more in test execution environments becomes unfeasible (Yoo and Harman 2012; Elbaum et al. 2014). Test case selection algorithms try to optimize the subset of all test cases that is executed, but according to Orso et al. (2004), efficient test selection techniques are "too imprecise and achieve little savings in testing effort", whereas "precise techniques are too expensive" when used with large systems. Plausibly, AI technologies and ML-based solutions should be capable of providing new alternatives, but so far AI- and ML-based software testing research has, according to the systematic literature reviews from Durelli et al. (2019) and Trudova et al. (2020), been focused on test case generation and test oracle generation.

E-mail addresses: torvald.martensson@liu.se, torvald.martensson@saabgroup.com.

https://doi.org/10.1016/j.jss.2025.112479
Received 8 December 2024; Received in revised form 1 March 2025; Accepted 25 April 2025
Available online 26 April 2025
0164-1212/© 2025 The Author. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1.2. Research question

In the companies the author works with as a researcher, practitioners have described how they use various methods and tools to test software units and components, but also how processes and descriptions of ways-of-working in the company are unilaterally focused on testing with traditional scripted test cases. Software developers articulate that there is a need for a structure for how different methods and tools contribute to different aspects of the quality of software units and components, and how methods and tools should be used in the best way. A structured and holistic approach seems to be missing in the companies, that includes all types of methods and tools used by practitioners.

In response to this, the purpose of this study is to investigate this field of research in a more structured way. The topic of this research paper is to answer the following research question: *How can companies developing large-scale and complex software systems evaluate and select methods and tools for testing of software units and components in a structured way?*

1.3. Contribution

The contribution of this paper is two-fold: First, it presents detailed results from workshops and focus groups with participants from six large-scale industrial companies, which give researchers and practitioners an improved understanding of what works (and what doesn't) in real industrial settings with regards to testing of software units and components. Second, it presents actionable strategies for how testing of units and components in a large-scale and complex software system can be conducted in a more structured way, which could help practitioners in software companies to increase test efficiency.

The remainder of this paper is organized as follows: The next section presents the research method. This is followed in Section 3 by a summary of the cross-company workshop. Section 4 presents the results from three focus groups, followed by a summary of the validation workshop with all six companies in Section 5 and a presentation of the results (three actionable strategies) from the validation workshop in Section 6. Section 7 presents the results from a review of related published research. Threats to validity are discussed in Section 8. The paper is then concluded in Section 9, including a description of further work.

2. Research method

2.1. A study in four steps

The research method for the study is based on recent guidelines for Grounded Theory (Birks and Mills 2023; Charmaz 2025) and lessons learned from Rodríguez (2024), i.e. reviewing the literature after building the theory, and validation and data triangulation to ensure trustworthiness of 'ground' data. An overview of the research method is presented in Fig. 1. The figure also shows how the six companies (further described in Section 2.4) were included in the different steps of the study.

The research study reported in this paper consists of four major steps:

- *Step 1: Cross-company workshop*: A workshop was arranged with 30 participants, from three companies in different industry segments (Section 3). The purpose of the cross-company workshop was to discuss how different methods and tools were used to test software units and components in the three companies (Company A, Company B and Company C). The workshop lasted for four hours, and was held at a physical location. The results from the cross-company workshop was a better understanding of how methods and tools for testing of software units and components work in conjunction and were used for different purposes in the three companies.
- *Step 2: Focus groups*: Three focus groups were arranged with a total of 19 participants from three other companies, also in different industry segments (Section 4). The purpose of the focus groups was to investigate the generalizability of the findings from the cross-company workshop, and to investigate how methods and tools were supported in ways-of-working and training in the three companies. The focus groups included eight participants (Company D), five participants (Company E) and six participants (Company F). Each focus group lasted for two hours and was held as an online video meeting. The results from the focus groups were a better understanding of generalizability, and of problems and experiences in the companies from using different methods and tools.
- *Step 3: Analysis and validation workshop*: The results from the cross-company workshop and the focus groups were analyzed with thematic coding analysis. The three themes identified in the analysis were presented at a validation workshop with 30 representatives from all six companies in the study (Section 5). The purpose of the validation workshop was to discuss and clarify ambiguities identified in the analysis, and to investigate the internal generalizability of the findings in the study. The workshop lasted for three hours, and was held as an online video meeting. The results from the validation workshop (Section 6) were three actionable strategies, which can be used to increase efficiency in testing of software units and components.
- *Step 4: Comparison with recent research*: The results from the workshops and focus groups were compared with recently published research (Section 7). The purpose of the comparison with recent research was to investigate the extent to which the methods and tools used in the companies were up to date with recent academic research, and to identify examples of methods and tools discussed in recently published research that were not discussed by the companies. From an extensive start set, 31 publications were selected for data extraction. The results from the comparison with recent research were increased confidence in the applicability of the results from the study, and a list of ideas for further work.

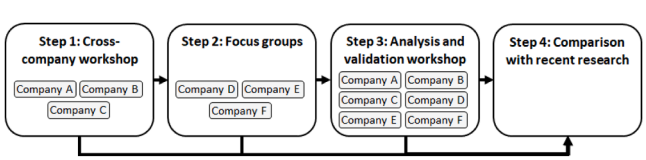

Fig. 1. An overview of the research method.

2.2. Larger workshops and smaller focus groups

The study is primarily based on workshops and focus groups with practitioners from six industrial companies. Originally, the word 'workshop' referred to a place where things are made or repaired. Today, workshop means an arrangement whereby a group of people learn, acquire new knowledge, perform creative problem-solving, or innovate in relation to a domain-specific issue (Ørngreen and Levinsen 2017). A focus group is generally defined as a group of individuals assembled by researchers (Powel and Single 1996) but usually as a smaller group, referred to as a small group discussion (Fern 1982) or as a group interview (Hughes and DuMont 1993). In this study, the size of the focus group was between four and twelve participants, as recommended by Brown (1999) if the group is homogeneous.

The study was purposely designed with two larger workshops (Step 1 and Step 3) and three smaller focus groups (Step 2) to allow method and data triangulation, following the guidelines from Rodríguez (2024) and Runeson and Höst (2009). The group format was chosen to allow the participants to explore and clarify their views in ways that would be less easily accessible in a one-on-one interview (Kitzinger 1995). According to Kitzinger (1995), when group dynamics work well, the participants work alongside the researcher, which is useful because "people's knowledge and attitudes are not entirely encapsulated in reasoned responses to direct questions". As we have seen in our previous work (Mårtensson et al. 2023), a large workshop with several participants

from each of the companies can enable constructive discussion, whereas the smaller focus group format allows in a better way for all participants to be heard, and the study was therefore designed to combine the two formats.

The participants at the workshops and the focus groups were selected as engineers working with testing, and methods and tools for testing of software units and components. Pre-defined questions were used at the workshops and focus groups to mitigate the problem of how groups in most cases tend to become influenced by one or two dominant people (Greenbaum 2003). When using the online format, special attention was paid to proactive strategies to build researcher-participant rapport, following the guidelines from Shamsuddin et al. (2021).

During the discussions, the researcher was acting as a moderator in order to involve representatives from all the participating companies. The discussions were summarized and quotes were transcribed in parallel with the ongoing discussions. In some cases, quotes were read back to the participants to ensure accuracy but without interrupting the interaction between participants, following the guidelines from Kitzinger (1995). The results from each workshop and focus group were analyzed based on thematic coding analysis as described by C. Robson and McCartan (2016), outlined in the following bullets:

- *Familiarizing with the data*: Reading and re-reading notes and transcripts, noting down initial ideas.
- *Generating initial codes*: Marking particularly interesting statements and coding in a systematic fashion across the data set.
- *Identifying themes*: Collating codes into potential themes, gathering all data relevant to each potential theme, and checking whether or not the themes work in relation to the coded extracts.
- *Integration and interpretation*: Structuring the data into a thematic map to support subsequent analysis, and assessing the quality of the analysis.

2.3. Reviewing recently published research

To compare the results from the workshops and the focus groups with recently published research (Step 4), a literature review was conducted. The questions driving the review were phrased as "Are the methods and tools presented at the workshops and focus groups in the study topics of recently published research?" and "What other methods and tools used to test a software unit or component are discussed in recently published research?" The literature review was not conducted as a complete and exhaustive systematic literature review according to guidelines from e.g. Kitchenham (2004). Instead, the review was designed to provide good examples of related academic publications, influenced by the guidelines for literature studies from Wohlin (2014):

- Define a *start set* with publications selected from the initial search, based on their relevance to the question driving the review.
- Conduct *iterations* including "backward and forward snowballing", i.e. examining the reference list of the publications in the start set, and publications citing the publications in the start set.
- Perform additional searches based on the *authors* of the included papers, and the journals and conferences where included papers are published.
- All papers selected to be included in the literature review go into *data extraction*, which should be conducted in accordance with the questions driving the review.

The proceedings from the IEEE/ACM International Conference on Software Engineering 2024 with its co-hosted conferences (ICSE 2024) and the proceedings from the IEEE International Conference on Software Testing, Verification and Validation 2024 with its co-hosted conferences (ICST 2024) were selected as a start set for the review, as these two venues in a good way represent recently published research related to software engineering and software testing.

2.4. Companies in the study

The study includes six large-scale industrial companies, referred to as Company A, Company B, Company C, Company D, Company E, and Company F. The companies operate in six separate industry segments:

- AI-powered software for autonomous driving
- Automotive products and services
- Communications systems and services
- Services and solutions for military defense and civil security
- Transport solutions for commercial use
- Video surveillance cameras and systems

Due to non-disclosure agreements, this publication does not refer to the companies in the study by name or specify the industry segment of each company. The companies included in the study are all multinational organizations, with the size of the organization ranging from 800 to >100,000 employees: two companies with >800 employees, two companies with >20,000 employees and finally two companies with >100,000 employees.

All companies develop large-scale and complex software systems for products, which also include a significant amount of mechanical and electronic systems (including standard components as well as bespoke hardware). The companies were purposely selected as suitable for the study, as they have similar characteristics but also operate in different industry segments.

2.5. Terminology

As always, some confusion exists around terminology, i.e. is the right wording "software component", "software module", "software unit", or something else. Myers et al. (2011), with the first edition of the book being one of the first books on software testing, uses the terms component testing, module testing, and unit testing interchangeably, and defines this as "a process of testing the individual sub-programs, subroutines, classes, or procedures in a program", which is distinct from "testing the program as a whole". In contrast to this, based on the xUnit notation (Meszaros 2007), Crispin and Gregory (2009) distinguish between unit and component tests: "Unit tests verify functionality of a small subset of the system, such as an object or method. Component tests verify the behavior of a larger part of the system, such as a group of classes that provide some service."

The International Software Testing Qualifications Board Standard Glossary of Terms Used in Software Testing includes definitions for software quality and testing terms used in the ISTQB certification products. The ISTQB Glossary V4.5 (ISTQB 2024) defines a component as "A part of a system that can be tested in isolation" and component testing as "A test level that focuses on individual hardware or software components." Furthermore, according to the ISTQB Glossary, module testing and unit testing are synonyms for component testing. In contrast to this, system testing is defined as "A test level that focuses on verifying that a system as a whole meets specified requirements."

This publication refers primarily to "testing of software units and components", as this reflects how the terms were used in the studied companies, acknowledging that participants at the workshops and focus groups in the study may or may not see a hierarchy between units and components. That is, testing of software units and components refers to testing of a part of a system that can be tested in isolation, i.e. not testing of the complete system.

3. Cross-company workshop

3.1. A cross-company workshop with three companies

The first step in the study was to arrange a cross-company workshop with 30 participants from Company A, Company B and Company C. The

purpose of the workshop was to discuss how different methods and tools contribute to the quality of a software module (units and components) in the three companies. All participants at the workshop had different types of experience from working with testing of software units and components in their companies.

Before the cross-company workshop, each of the three companies were asked to prepare two or three presentations of a method, a tool or a test technique contributing to the quality of software modules in their company. At the cross-company workshop, presentations from the companies were mixed with open discussions among the workshop participants based on the following pre-defined questions:

- Why do we use the presented methods and tools, and not scripted test cases and code coverage?
- Are the presented methods and tools easy or difficult to use and understand for most engineers?
- Should the presented methods and tools be used by all engineers or a central function?
- Do the presented methods and tools primarily provide feedback or measure quality?
- Should the presented methods and tools be used early or late in a development cycle?
- Are any of the presented methods and tools more valuable for particular types of software?

The set of questions was created primarily based on the researchers experience from previous work (Mårtensson et al. 2019; Mårtensson et al. 2023). The set of questions was designed to promote constructive discussion among the workshop participants.

3.2. Presentations at the cross-company workshop

At the workshop, the companies held short presentations in the following order:

- "Fuzzy testing with RESTler-fuzzer" (Company A):
- "CodeScene - intro, status and early findings" (Company B)
- "Industrial application of mutation testing" (Company C)
- "Experiences from using sanitizers and Valgrind" (Company C)
- "Multi-component testing – an overview" (Company B)
- "Testing of configurable and distributed software components" (Company C)
- "Performance testing of components with hardware systems" (Company A)

Each presentation was followed by questions from other workshop participants, and open discussion. The presentations were summarized (as input to Step 2 in the study) and each summary was sent to the presenter after the workshop for approval:

"Fuzzy testing with RESTler-fuzzer": RESTler-fuzzer is a stateful fuzzer from an open source project, used to fuzz REST APIs. It analyzes open APIs and then generates and executes tests that exercise the service though its REST API. Pros identified by Company A were: 1) Good complement to function tests, identifying other types of problems. 2) Less coding effort than implementing function test cases. 3) Considered as a good starting point for new REST APIs. In addition to this, cons identified by Company A were: 1) A bit finicky to get to work. 2) Documentation not considered to be very good. 3) Lacks some functionality, e.g. lacks ability to clean up created resources. Company A introduced RESTler-fuzzer to improve test coverage, and to detect issues that traditional function testing might miss. Company B planned to encourage developers to use the RESTler-fuzzer, in particular before functional tests were in place. So far, the bugs identified by RESTler-fuzzer were related to input validation (but so far based on very limited experience).

"CodeScene - intro, status and early findings": CodeScene is an ML-based set of tools from the company with the same name, which analyzes code in Git and analyzes trends. As used by Company B, input includes all commits with historical data, Jira tickets and team setup. Results include hotspots, code health, architectural coupling, defect distribution and team collaboration. Code health is a score (1–10) based on 26 code smells. CodeScene is a language neutral tool that can help increase understandability of the code. The tool does not include code coverage or checking of requirements. The tool was well received by the developers at Company B, describing it as easy and flexible to use. Company B's experience so far was that CodeScene could help in practice to prioritize areas for refactoring. Company B had also observed a correlation between code health and number of problem reports from the customers.

"Industrial application of mutation testing": Mutation testing evaluates the adequacy of existing software tests. Mutation testing involves modifying a program in small ways (creating a mutant) that mimic typical programming errors. Test suites are measured by the percentage of mutants they kill, i.e. their ability to detect errors. Mutation testing is based on the idea of the competent programmer and the coupling effect (test cases that detect simple types of faults are sensitive enough to detect more complex types of faults). Company C's reported experiences from using mutation testing: Results from mutation testing were, according to the developers, providing actionable information. The feedback mutation provided to the developers was described as more important than measuring the mutation score. With improvements of the tools used for mutation testing, execution time was now considered to be acceptable. Equivalent mutants and unproductive mutants (both described as problematic in related literature) were not seen as important problems in Company C.

"Experiences from using sanitizers and Valgrind": Sanitizers instrument a program at compile time with checkers, and detect suspicious behavior as the program runs. There are different types of sanitizers, such as Address Sanitizers (ASan), Undefined Behaviour Sanitizers (UBSan), Memory Sanitizers, and Leak Sanitizers. Sanitizers can identify problems that are difficult to find with functional test cases, such as memory leaks, out of bounds array indexing, and undefined variables. Sanitizers depend on a test suite or a binary that execute the interesting parts of the program. The better coverage the test suite (or the binary) has the more value the sanitizers provide. Company C described the value provided by sanitizers as: 1) Amplifies existing software test suites. 2) The sanitizers allow the test suites to provide more value and find more faults. 3) Easy to set up (but could be hard to understand). Valgrind was described as more competent than sanitizers as a memory debugging tool, but as much slower than sanitizers.

"Multi-component testing – an overview": Multi-component testing in Company B is testing of two or more components in a large-scale software system. A multi-component in Company B includes millions of lines of code. The test cases for a multi-component in Company B are interacting with the multi-component in the same way as other test cases test each of the components (i.e. multi-component testing is not integration testing). The test cases are executed in a host environment, i.e. not on the same hardware as in the target (production) environment. Company B's experience was that multi-component testing is an efficient way to enable large quantities of testing, with exceptions for some types of testing such as testing of algorithms (which require testing on component level). One observation was that the developers sometimes tended to focus too much on the multi-component tests, leaving out the component tests that are also needed.

"Testing of configurable and distributed software components": Company C presented testing of configurable and distributed software components in the platform layer of a large-scale and complex software system. The system included software components in a platform layer and, on top of that, application software components. The platform layer consisted of a number of generic components, distributed to the different subsystems. The behavior of the platform component was dependent on configuration parameters provided as input to the components. In order

to test the components in the platform layer, Company C replaced the application software components with generic test code components with pre-defined behaviors. Company C described it as easy to generate new test code components after system changes, and to produce a new test suite to test a new system configuration. Because of that, the presented approach was a good way to find problems as early as possible, and also a more efficient way to test the components than extensive component tests or system tests.

"Performance testing of components with hardware systems": Company A presented performance testing of components in a test environment with the same hardware as in the production environment. The tested software component was ML-based software based on a neural network. In this type of performance testing, Company A measured response time on the component API when the component accessed the same type of hardware systems as those used by their customers. Before this type of testing was introduced at Company A, there were repeated problems wherein performance had degraded with no one knowing or taking action. Now this test activity had been introduced into the continuous integration flow, and the organization was immediately aware of any problem. Company A described that it was an efficient way to conduct this type of testing on component level, and that this made it possible to avoid surprises with regards to performance in their product.

3.3. Discussions at the cross-company workshop

At the workshop, the researcher was acting as a moderator, involving representatives from all three companies in each of the topics that were discussed by the participants. Questions from the participants and open discussions were mixed with predefined questions.

The workshop participants discussed the questions in smaller groups, followed by open discussions with all participants. The participants reached consensus around the following descriptions of why the presented methods and tools are used: *Fuzzy testing* is used to get started quickly and efficiently, as a complement to functional test cases. *CodeScene* is used to decrease technical debt, increase understandability of the code, and to get a good overview of status for the code base. *Mutation testing* builds confidence for the test cases in the test suite, emphasizes problem areas, and removes unneeded test cases. *Valgrind and sanitizers* find problems that are difficult to write test cases for, and are an inexpensive way to find complicated problems. *Multi-component testing* is used to increase efficiency in component testing, providing more value for the time/money. *Testing distributed components* is used to test more things at component level, and is more cost-efficient than testing at the system level. *Performance testing with hardware* is used to identify hardware-related problems with testing at the component level. Fig. 2 presents a visualization of the researcher's interpretation of the participants' responses at the cross-company workshop.

Some of the presented methods and tools were considered to be easy to use and the results easy to understand, while others required knowledge and experience from senior engineers: *Fuzzy testing* and *Multi-component testing* were considered to be easy to use and understand. In contrast to this, the workshop participants agreed that *Valgrind, Mutation testing, Testing distributed components*, and *Performance testing with hardware* require knowledge and experience from a senior engineer. Results from *CodeScene* and *sanitizers* were generally seen as easy to understand, but some aspects require experienced engineers. According to the workshop participants, methods and tools considered to be easy to use and understand can be used by all engineers, while the others require support from a central function.

Multi-component testing and *Testing distributed components* were considered as most relevant to measure quality, while the other presented methods and tools were considered to be most relevant to provide feedback to developers. All presented methods and tools were considered as relevant to use continuously, i.e. not only early or only late in a development cycle. One participant stated that using sanitizers provided the "most bang for the buck", a comment for which several other participants expressed their support.

When asked if the presented methods and tools are more valuable for some types of software, the workshop participants started discussing how there is a constant need to test more efficiently in the software industry, and how we must try to find better ways to cut costs and save time, and identify problems earlier. The cost drivers for testing were described as systems with many different configurations (including hardware and software configurations), and large-scale systems (with many components and many system functions). According to the participants, the presented methods and tools were considered as more valuable for software with data-driven behavior, software with a lot of dependencies between software modules, and large-scale software systems.

3.4. Summary and analysis

At the cross-company workshop with Company A, Company B and Company C, the companies presented experiences from using different methods and tools. The discussions among the workshop participants revealed that the methods and tools were used for different purposes,

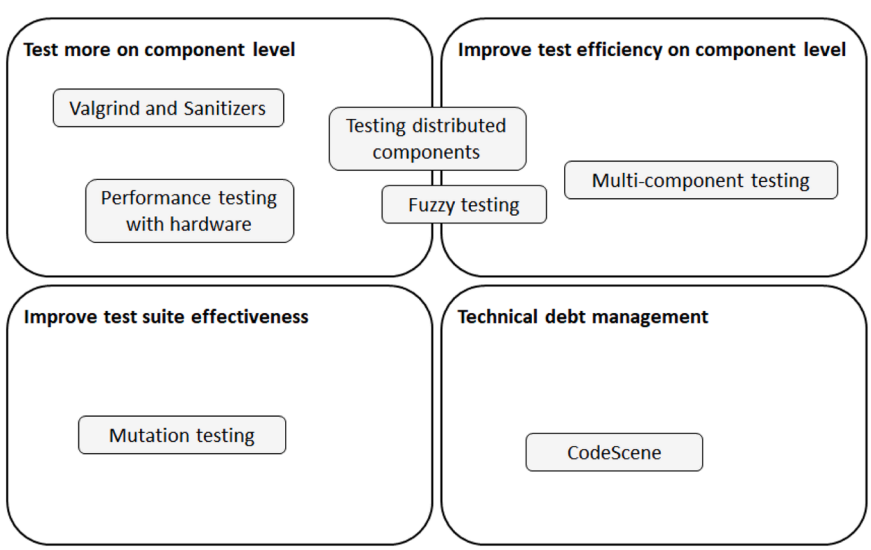

Fig. 2. Methods and tools used for different purposes within Company A, Company B and Company C.

resulting in four categories (visualized in Fig. 2). Different methods and tools are not mutually exclusive but rather work in conjunction. Some of the presented methods and tools were considered easy to use, while others were seen as more difficult, requiring support and training.

According to the participants at the cross-company workshop, the presented methods and tools were considered to be more valuable for software with data-driven behavior, software with a lot of dependencies between software modules, and large-scale software systems. The participants at the workshop also discussed the constant need to test more efficiently in the software industry, and how we must try to find better ways to cut costs and save time, and identify problems earlier. As scale and complexity are cost-drivers for testing, *testing of software units and components is an increasingly demanding task*, as it requires knowledge of many different types of methods and tools. This insight motivated the next step of the study, to investigate the generalizability of the findings, and to investigate how methods and tools were supported in ways-of-working and training in large-scale organizations.

4. Focus groups

4.1. Focus groups with three companies

The next step of the study was a series of focus groups with Company D, Company E and Company F. The purpose of the focus groups was to provide insights from companies in other industry segments, and investigate how methods and tools are described in ways-of-working and courses or training programs in the companies, which informed the list of pre-defined questions used at the focus groups. The focus groups included engineers with experience from different types of testing of software modules (unit and components) from different teams in their companies.

The summaries of the presentations at the cross-company workshop (Section 3.2) and a summary of the discussions at the cross-company workshop (Section 3.3) were sent to the participants in each of the focus groups two days before the focus group, to allow the participants to reflect and prepare. At the focus group, presentations of the material were mixed with the following predefined questions:

- Are the engineers in your company using any of the methods and tools presented at the cross-company workshop?
- What other methods and tools are used within your company to test a software module (units and components)?
- Are all methods and tools used by the engineers in your company described in your processes and/or ways-of-working?
- Are all methods and tools used by the engineers covered by courses or training programs in your company?

4.2. Focus groups with company D

The focus group with Company D included eight participants. The participants described how they recognized all of the presented methods and tools from the cross-company workshop (Section 3.2). Most of the presented methods and tools were used in Company D, but Company D did not use anything similar to *Testing distributed components* or *Performance testing with hardware*. Company D described how, in addition to CodeScene, they also used other tools such as QAC and Clang Tidy for the same purpose. With regards to *Multi-component testing*, Company D described how they tested with other components referred to as libraries. The participants also described how they used Bazel to increase test efficiency, by testing only modules that have changed.

Responding to the question about how methods and tools were described in processes or ways-of-working, the focus group participants did not express any positive opinions or comments. One of the described problems was that "documents were outdated fast", and therefore needed to be short and easy to update. The lack of structure was described as "very problematic", due to the fact that the company had many new employees. One participant stated that it is "very unclear what you are expected to do and how to get help". The participants expressed similar opinions in response to the question about courses and training programs, with one engineer concluding that "training and courses does not work very well". Another participant commented that instead of organized courses, support was dependent on how "senior engineers help younger engineers".

The focus group participants also discussed how this affected the organization. The participants seemed to agree that in some cases the developers used the tools, but without understanding the reason and purpose. One participant stated that methods and tools can be of great benefit but can also be seen as problems: "We need many tools, but everything the developers must comply to is a burden for the organization". Another participant summarized: "Developers do not like constant CI failures. The feedback must be understandable, actionable and explain why it is important – otherwise the developers will quit."

4.3. Focus groups with company E

The focus group with Company E included five participants. Similar to the Company D focus group, the participants in the Company E focus group described how they recognized the presented methods and tools from the cross-company workshop (Section 3.2). Many of the presented methods and tools were used in Company E, but Company E did not use anything similar to *Testing distributed components* or *Performance testing with hardware*. Company E did not use the tool CodeScene, but the participants described how they used static code analysis for the same purpose, together with other tools that check for constructions not allowed in the software. Company E also used BlackDuck, scanning the software for code owned by other organizations. Company E used *Mutation testing* and another similar method the participants chose not to describe further. The participants described how they used Model Advisor, Polyspace and GCC as *sanitizers*. The participants also described how Company E used tools for testing of security and other specific requirements.

The focus group participants were also asked how methods and tools were described in processes or ways-of-working. One participant described how "the way we work is not well described, we have no test strategy". According to the participants at the workshop, knowledge about the company's ways-of-working was primarily transferred from one engineer to another. The participants described how the company had consultants from the tool suppliers, providing courses and support, but that the training and support was "from the tool's perspective and not how to actually utilize the tool" in the company.

The focus group discussed how the way the tools were used in the company was causing problems. To quote one participant: "The first thing you do as a new developer is to install a giant pile of tools, that you do not have a clue how you should use". One of the participants described plans for how tools should be integrated in an automated pipeline, to provide feedback in a more structured way. The focus group also discussed how "there is a constant need to be more efficient", and how this affected the use of methods and tools. One participant described how the company was aiming at streamlining the use of different tools, to use a smaller number of tools in the company.

4.4. Focus groups with company F

The focus group with Company F included six participants. The participants in the Company F focus group recognized all the presented methods and tools from the cross-company workshop (Section 3.2) except for *Mutation testing*. The participants described how they used *Fuzzy testing* (in a smaller context), *Multi-component testing, Sanitizers and Valgrind*. Company F did not use CodeScene, but used CodeClimate for the same purpose. Company F did not use mutation testing, but worked with different aspects of test coverage. The participants explained how they did not use anything similar to *Testing distributed components* as they

did not "have this type of architecture". Company F did not use *Performance testing with hardware,* but tested components with a simulator model representing the hardware. Company F described how they used tools to test cybersecurity (e.g. dependency scan or container scan) and domain-specific tools to test specific requirements.

The participants at the workshop with the Company F focus group discussed how it was difficult to introduce a new tool in the organization, and how the tools often were not yet described in the company's processes. According to the participants, in some cases there were descriptions of how for example tools for static code analysis should be used, but the descriptions were not yet in the company's processes. Some things, e.g. how sanitizers should be used, were according to the participants not described at all. The participants described how training must be "at the right time" to not be a burden, and how it was difficult to find that timing. One participant stated: "Training or courses for how to use the tools? No, generally we do not have that."

The participants discussed how methods and tools caused frustration in the company, as the tools were also used to set rules for software committed to the main branch. One participant stated: "Developers react 'I did not know about this' when they are notified by static analysis tools about violated coding rules". The lack of clearly described ways-of-working also seemed to cause confusion, exemplified in for example the comment "Something that takes time from the build is generating friction. Developers do not understand why we have the tools."

4.5. Summary and analysis

At the focus groups with Company D, Company E and Company F, presentations of the results from the cross-company workshop were mixed with predefined questions. All three focus groups recognized all the presented methods and tools from the cross-company workshop, with one exception for mutation testing in Company F. Fig. 3 visualizes how methods and tools presented at the cross-company workshop were also used within Company D, Company E and Company F.

The figure shows how all companies utilized methods and tools in each of the four categories which strengthens the generalizability of the findings from the cross-company workshop. According to the focus group with Company F, the company did not use mutation testing but improved the test suite with other methods for test coverage (without exactly revealing how). CodeScene was used only by Company D, but all three companies described how they used tools for static code analysis (e.g. Code Climate, Clang Tidy, QAC) for the same purpose.

The participants in the three focus groups described different types of problems related to feedback from tools, and how the developers did not understand the feedback or why it should be considered as important. The participants in all three focus groups also described how methods and tools used within their company were often not well described in the company's ways-of-working (processes etc.). In a similar way, the participants in all three focus groups described how there was a need for support and courses. This caused frustration and confusion, which *indicates a potential to increase test effectiveness and efficiency, if methods and tools used for testing of software units and components could be used in a more structured way* in each of the companies.

5. Analysis and validation workshop

5.1. Analysis

The discussions at the cross-company workshop and the focus groups were summarized and quotes were transcribed in parallel with the ongoing discussions. The results were analyzed in an iterative process based on thematic coding analysis (Section 2.2) as described by C. Robson and McCartan (2016), resulting in three major themes:

- *The four categories of methods and tools*, representing test activities with methods and tools for different purposes. This theme was based primarily on the presentations and the discussions from the cross-company workshop and the focus groups (summarized in Section 3.4 and Section 4.5).
- *Missing ways-of-working and training*, which caused frustration and confusion in each of the companies. This theme was based primarily on quotes such as "training and courses does not work very well" from the three focus groups (summarized in Section 4.5).
- *Problems with feedback*, which also caused frustration and confusion in the companies. This theme was built primarily on descriptions of "CI failures" and quotes such as "developers do not understand why we have the tools" from the focus groups (summarized in Section 4.5).

The thematic coding analysis also identified ambiguities and potential differences in the descriptions by participants from different companies. These findings were used to inform the list of pre-defined questions for the validation workshop.

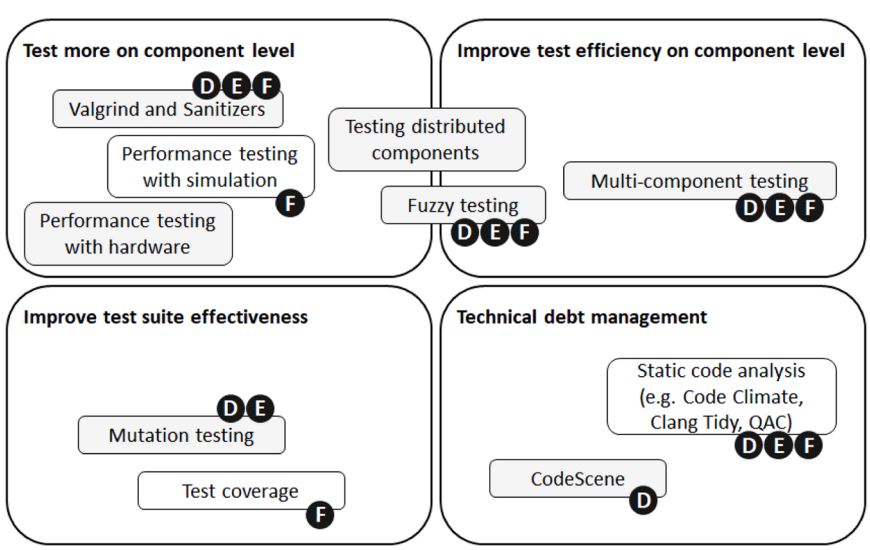

Fig. 3. Methods and tools presented at the cross-company workshop used within Company D, Company E and Company F (marked with the letters D, E and F).

5.2. Validation workshop with all companies

The last step of the study was a validation workshop with all companies in the study (Company A, Company B, Company C, Company D, Company E and Company F). The purpose of the validation workshop was to validate the internal generalizability of the findings from the previous steps. The validation workshop included 30 engineers representing all six companies in the study. The participants were selected from the companies to include both participants who had been part of the previous steps, and also individuals not previously involved in the study, in order to add new perspectives. All participants were selected as engineers working with testing, and methods and tools for testing of software units and components.

At the validation workshop, presentations of the findings from the cross-company workshop (Section 3) and the focus groups (Section 4) were mixed with predefined questions. The questions discussed at the validation workshop with all companies were selected to clarify ambiguities identified in the analysis of the results from the cross-company workshop and the focus groups:

The companies use different tools for maintainability and understandability, in what ways are those tools similar and how are they different?

- What is multi-component testing and how is it different from system testing or integration testing?
- What are the pros and cons of including tools in the continuous integration and delivery pipeline?
- What are the success factors when introducing a new tool into the organization?
- Are there other (better) ways [than the visualization in Fig. 2] to sort methods and tools into categories?
- The presented methods and tools are described as enablers, so why are they also a source of frustration?

5.3. Maintainability and understandability

At the three focus groups, participants from all three companies described how they used tools other than CodeScene for the same purpose (as described in Sections 4.2–4.4). The participants at the validation workshop were asked about the ways in which the tools (CodeScene, CodeChecker, Code Climate, Clang Tidy and QAC/QAC++) are similar and how they are different.

The discussion reached a consensus that all the tools are used to increase maintainability and understandability of the code, and they "can tell things about the code but in different ways" (quoting one of the participants). An individual from Company C commented "CodeChecker is a more active tool for developers. Day-to-day basis for the developers and team. You can use it to improve the developers, not just improve the code. This can be of more value." Another participant from Company D commented: "CodeScene must be used in a different way as it is not rule-based, it's more difficult to act on. Teams tend to use the rule-based part of CodeScene."

5.4. Multi-component testing

Company D, Company E and Company F all described how they used test approaches similar to multi-component testing, as the concept presented by Company B (Section 3.2). At the validation workshop, Company B was asked to present the concept again, in order to investigate whether or not the companies used similar types of testing or if this was merely a confusion of terminology. A participant from Company B described e.g. how this was "Not really different from testing a subset of the system. It is more of the software scale." and "a good way to test a huge amount of functionality fast".

After a brief discussion, the participants found that all companies seemed to use similar approaches. One individual from Company B emphasized that multi-component testing is not a silver bullet for testing of software components, but "one of the tools in the toolbox". The participants also discussed best practices for selecting the multi-component in order to keep the interfaces stable. One individual from Company D stated that "We see many similarities when we look into companies in different domains when it comes to testing a software component" and added "We should discuss this more, how to test on component level."

5.5. Tools in the continuous integration and delivery pipeline

The participants at the validation workshop were also asked if the presented tools should be used in a continuous integration and delivery pipeline, similar to the automated pipeline described by Company D including QAC, Clang Tidy, sanitizers, Valgrind and Bazel (Section 4.2). One individual from Company C stated that the tools should be used to monitor technical debt: "You do not want to add technical debt, we had a lot warnings but should not add new warning. You should at least not make it worse." A participant from Company E added "Applying sanitizers and Valgrind to an existing code base would create problems. If you have used the tools from the beginning, you would not have problems."

An individual from Company F presented a different view, stating that "a CI pipeline should not stop if you have technical debt". Another participant from Company D commented that "Fulfilling a code standard can be a requirement". Another voice from Company C described how all checks can be bypassed: "A check for cyclomatic complexity is great, but I have seen developers [changing the software to] simply bypassing the check instead of writing understandable code."

5.6. Introducing new tools into the organization

Company B and Company F described very differently how a new tool was received by the developers. Company B described how CodeScene was well received by the developers, describing it as easy and flexible to use (Section 3.2). In contrast to this, Company F described how developers "resist like crazy" when a new tool was introduced (Section 4.4).

Based on this the participants at the validation workshop were asked to discuss the success factors when introducing a new tool in the organization. A company C representative stated: "We need good examples, we should not say 'use this tool because I said so'. We must explain how to avoid certain types of problems, and motivate with examples. Myself, I can be quick to dismiss something before I see what a tool can actually do." Other participants agreed on the idea to "lead by example" and added that there needs to be a "pain that you take away" (a problem that the tool is solving).

An individual from Company E commented that you should focus on the first 20 % early adapters and grow from there ("20 % are prepared to take in a new tool, 60 % are skeptical or reluctant, and 20 % are really negative but you should not listen to them").

5.7. The visualization of methods and tools

At the focus group with Company F (Section 4.4), the participants commented on the visualization of the cross-company workshop participants' response (Fig. 2 in Section 3.3) as "this makes sense" and "one way to slice the cake". The representatives from the six companies at the validation workshop were asked to discuss other (better) ways to sort methods and tools for testing of software modules into categories of some type.

The participants started with a discussion on how to know when enough testing has been done ("Are we testing too much? How do we know that?") and if tools can help to answer that. One of the participants commented that this is related to the term test diversity (Nikolik 2006) in research. The participants agreed that the categories in Fig. 2 (in

Section 3.3) were relevant and sound, but several voices questioned if "Test more on component level" was a good name.

The participants then continued to discuss why one might want to test more on component level: One participant simply stated that we "use Valgrind and sanitizers so we can sleep at night". Other voices described the problem with developers having "CI exhaustion", waiting for feedback from test activities in a continuous integration and delivery pipeline, and the need to test more on the component level (aspects not found with test cases) in order to provide fast feedback to the developers. The participants discussed if a better name for "Test more on component level" should be "Enhance quality on component level (shift left)", "Improving testing on component level" or "Shorter feedback loops", but agreed on the name "Optimize feedback loops" as the best alternative.

5.8. Tools as an enabler or as a burden

The companies at the cross-company workshop discussed how there is a constant need to cut costs and to identify problems earlier, and how many configurations and large-scale systems are cost drivers (Section 3.3). Company A, Company B and Company C described the presented methods and tools as enablers, increasing test effectiveness and test efficiency (Section 3.2), which showed in for example the quotes "multi-component testing is an efficient way to enable large quantities of testing" (Company B) or "Sanitizers can identify problems that are difficult to find with functional test cases" (Company C).

In contrast to this, when asked to discuss ways-of-working and training, Company D, Company E and Company F described how tools are seen as a burden for the organization, causing confusion and extra work (Sections 4.2–4.4), showing for example in the quotes "We need many tools, but everything the developers must comply to is a burden for the organization" (Company D) or "Developers react 'I did not know about this'" (Company F).

The participants at the cross-company workshop were asked to discuss this contradiction. One voice commented that a big problem is "false positives, tools nagging you for problems that is not a problem". Another participant from company C stated: "One source of frustration is that tools are seen as they delay the developer, it is about a context switch. Feedback is separated from when you commit. It takes too long time from commit to feedback. You start thinking about something else." Another participant from Company E commented: "The most frustrating is that I have to use a tool and I do not understand why. Why must I fix these warnings, to follow this rule. Then it is a big burden." Another voice added: "Some people are happy to use a tool, thinking that this tool helps me check things so I do not have to think about it. Others see feedback as complaints. It has a lot to do about different personalities."

5.9. Other reflections from the participants in the study

The participants concluded their discussions with reflections on how the findings could be applied at their companies. A representative from Company C stated that an actionable takeaway from the study was that "This is about people, how people react to new things. This is about onboarding people. This is what it is all about, even though that software development is a very technical subject." An individual from Company E commented that it is "always good to know that we are not the only ones that are lost, others have the same pain points", a statement that several other participants agreed with.

A participant from Company B stated that the main takeaway and a useful rule of thumb was that if a developer reacts by asking "why do I need to do this" when using a tool, the organization has a problem that needs to be fixed. A participant from Company E concluded that "We need to motivate why we use a tool, we need a structure for this. We need a clean statement for why we use tools."

5.10. Summary and analysis

At the validation workshop with all six companies (Company A, Company B, Company C, Company D, Company E and Company F), presentations of the findings from the cross-company workshop and the focus groups were mixed with predefined questions and open discussions. At the validation workshop, the participants from the six companies were asked to discuss how different tools were used to increase maintainability and understandability, how the companies used similar approaches for multi-component testing, and how tools could be included in a continuous integration and delivery pipeline. The participants at the validation workshop generally confirmed the findings from the cross-company workshop and the focus groups, and raised no new questions related to reliability or generalizability of the findings.

The participants at the validation workshop also discussed *the four categories of methods and tools* for testing of software units and components (visualized in Fig. 2 and Fig. 3). The discussions did not include any hesitation or critique related to the four categories. However, the discussions resulted in a suggestion for a new name for one of the categories, which improves the presentation of the proposed structure with the four categories.

The participants at the validation workshop discussed the need for *support in ways-of-working and courses* or training programs. Ways-of-working and training must be tailored, depending on whether a tool is used by a few experts or by all developers, and balancing support based on the tool itself and how the tool is used in the organization.

The participants at the validation workshop further emphasized the importance of *feedback to developers*. Results from some test activities (involving tools such as Valgrind) can be difficult to interpret and understand, which cause confusion and frustration among developers. Therefore, feedback from the continuous integration and delivery pipeline must be understandable (describing what the problem is), actionable (describing what the developer needs to do) and explained (describing and motivating why this is important).

The participants at the validation workshop generally confirmed the findings from the cross-company workshop and the focus groups, strengthening the internal generalizability of the findings within all the six different companies. In summary, the results from the cross-company workshop, the focus groups and the validation workshop form three actionable strategies, which are further described in Section 6.

6. Results and discussion

6.1. The iterative process of creating and evaluating theories

Based on the constructivist approach to Grounded Theory (Charmaz 2025), this study has utilized an iterative process of creating and evaluating theories throughout the steps described in Section 2 and illustrated in Fig. 4.

A theory is defined by Thornberg and Charmaz (2011) as "A theory states relationships between abstract concepts and may aim for either explanation or understanding". The term 'theory' remains slippery in grounded theory discourse and mirrors ambiguities about what theory means (Charmaz 2025). Bryant (2009) asserts that original grounded theories need not be conjectural or universally explanatory, rather they are contextually constrained awaiting further development. Based on guidelines from Birks and Mills (2023), the report of the study is

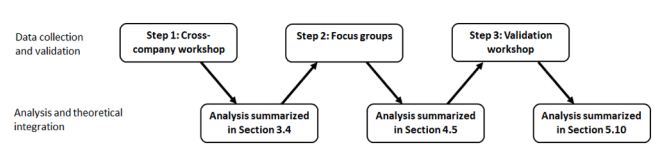

Fig. 4. The iterative process of creating and testing theories in the steps of the study.

concluded with a series of recommendations, presented here as actionable strategies. The actionable strategies in Section 6.2 are described as practical steps that can be implemented to achieve a desired goal or outcome, allowing organizations to take concrete actions. The three actionable strategies are based on the analysis of the cross-company workshop, the focus groups and the validation workshop with all six companies (summarized in Sections 3.4, 4.5 and 5.10).

6.2. Three actionable strategies

Practitioners in the software industry describe how there is a need to cut costs and save time, and to identify problems earlier. More components and more configurations, as well as more dependencies between software modules and data-driven behavior are cost-drivers for testing, calling for other methods and tools to complement traditional scripted test cases. In response to this, the findings from this study provide three actionable strategies (visualized in Fig. 5) for how testing of units and components can be conducted in a more structured way, which can be used to increase test efficiency:

1. *Evaluate and select methods and tools based on the four categories*
2. *Finetune the feedback from the test activities in the pipeline*
3. *Provide the right support in ways-of-working and training programs*

(1) Evaluate and select methods and tools based on the four categories, representing test activities with methods and tools for different purposes: As every tool is a cost for the organization, there is a need for a structured and holistic approach to how methods and tools should be used. The four categories provide a structure for how methods and tools must be evaluated and selected to complement each other in a structured way, to be used for different purposes and to work in conjunction:

- *Optimize feedback loops*: Providing fast feedback to developers on aspects otherwise tested at system level, with methods such as performance testing of components with hardware, or tools such as Valgrind and sanitizers.
- *Improve test efficiency*: Testing a huge amount of functionality faster and at a lower cost than scripted test cases on a component level, with methods and tools such as fuzzy testing or multi-component testing.
- *Improve test suite effectiveness*: Improving the adequacy of the test suite for a component and measuring the test suite's ability to find all defects, with methods such as mutation testing.
- *Technical debt management*: Providing a good overview of status for the code base to improve the maintainability and understandability of the code, with tools such as CodeScene, Clang Tidy or QAC.

(2) Finetune the feedback from the test activities in the pipeline: Results from some test activities can be difficult to interpret and understand, in particular from test tools in the continuous integration and delivery pipeline, which can cause confusion and frustration among developers. As feedback is separated (in time) from when you commit, the feedback from test activities in the continuous integration and delivery pipeline must be *understandable* (describing what the problem is), *actionable* (describing what the developer needs to do), and *explained* (describing and motivating why this is important).

(3) Provide the right support in ways-of-working and training programs for all methods and tools used for testing of software units and components: If methods and tools used within a company are not well described in the company's ways-of-working (processes etc.), this often cases frustration and confusion in the organization. To increase test effectiveness and efficiency, ways-of-working and training must be tailored differently for *tools used by a few experts* versus *tools used by all developers.* It is also important to balance support for *the tools itself* and *how the tool is used* in practice within the organization.

As software engineering is a constantly emerging field, the four categories should not be seen as static, but instead need to be refined and expanded, as new challenges such as AI generated software are emerging and affecting the testing of software units and components. However, as the proposed four categories are based on the workshops and the focus groups with the six studied companies, combined with the diverse nature of the six companies, it is reasonable to expect that the four categories can be applied to a large segment of the software industry.

The methods and tools presented at the cross-company workshop and the three focus groups were organized into four categories, representing different purposes, showing how different methods and tools complement each other and work in conjunction. It is plausible that the presentation of the four categories could be further improved, or that further work could expand the structure to include new categories representing other types of testing.

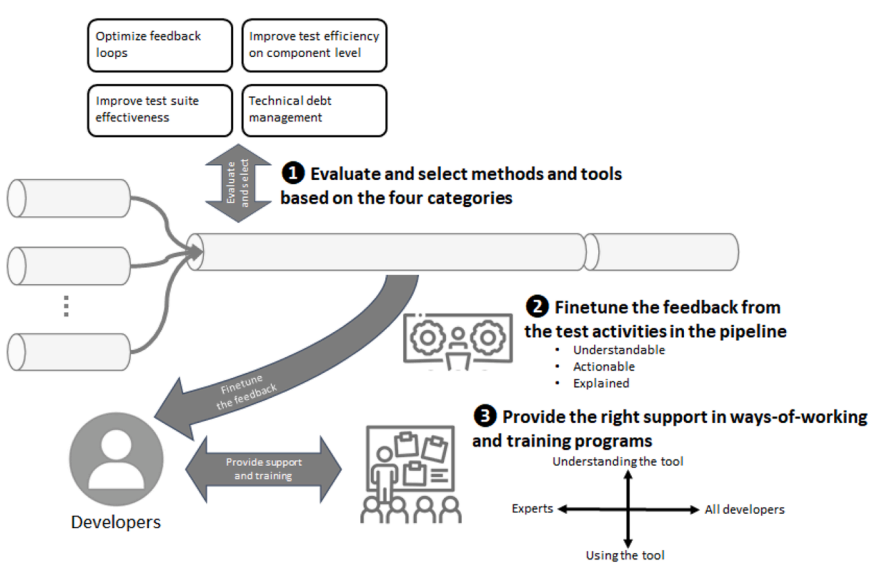

Fig. 5. Three actionable strategies for how testing of units and components in the continuous integration and delivery pipeline can be conducted in a more structured way, which could help companies to cut costs, save time and identify problems earlier.

6.3. Discussion

From the results of this study, we learn that successful implementations of a test strategy for testing of software units and components is not just dependent on the evaluation and selection of a good set of methods and tools. It is also very much dependent on how the selected tools are supported in ways-of-working and training programs, and on how tools are successfully integrated into the continuous integration and delivery pipeline. Quoting one of the participants at the validation workshop, successful implementations of methods and tools "is about people, how people react to new things". Because of this, success is based not only on technical knowledge about what works in software development and software testing, but also on change management and knowing the developers in the company.

The results from this study are related to our previous work (Mårtensson 2019), in which we analyzed the test activities in the continuous integration and delivery pipeline in four large-scale companies, and proposed a model for how the pipeline can be designed to support four stakeholder interests: "Check changes" (e.g. software developers in a development team or a function team), "Secure stability" (e.g. test managers or QA department), "Measure progress" (e.g. project managers or line managers) and "Verify compliance" (e.g. product owner or technical managers). As unit and component tests are primarily used to provide feedback to developers, it is reasonable to expect that developers are also the primary stakeholder for the methods and tools discussed in this study. However, as unit and component tests in for example the aeronautics or pharmaceutical industries are used to verify compliance with requirements, a product owner or a technical manager could also be a potential stakeholder. At the validation workshop, participants from two of the companies discussed whether technical debt should be included in an automated pipeline, which implies that e.g. a QA representative could or could not be a stakeholder for methods and tools related to technical debt.

One way of structuring test activities or results from test activities could be to relate this to established terminology within information retrieval (Powers 2011). For example, precision, as the fraction of relevant instances among the retrieved instances, corresponds to a test suite's ability to find only real defects (and not false positives), and recall, as the fraction of relevant instances that were retrieved, corresponds to the test suite's ability to find all defects (test suite effectiveness). Further research could investigate if this well-established structure and terminology could for example clarify cause and effect or provide other new insights.

The participants at the cross-company workshop, the focus groups and the validation workshop did not particularly emphasize new challenges related to AI-generated code. This could be seen as surprising, as all six companies in the study define themselves as software-based companies with clear strategies for AI and ML based solutions in their products. However, the companies described how they used tools that scan the software for code owned by other organizations, which could or could not come from an LLM trained on source code from other organizations. Generative AI for software development is a promising field of research, including generation of source code as well as test cases, but it also implies new questions and new challenges.

7. Comparison with recent research

7.1. Reviewing recently published research

To compare the results from the workshops and the focus groups with recently published research, a literature review was conducted. There were two purposes behind conducting the review:

- To investigate the extent to which the methods and tools used in the companies were reflected in the topics of recent academic research
- To identify examples of methods and tools discussed in recently published research that were not discussed by the companies

The proceedings from ICSE 2024 (with co-hosted conferences) and the proceedings from ICST 2024 (with co-hosted conferences) were selected as a start set for the review. As a first step, the title and then the abstract of all publications were reviewed manually to identify candidates corresponding to the questions driving the review. From this quite extensive start set, 31 publications were selected for data extraction. The full-text of the 31 publications were then manually reviewed, and in some cases references or related publications were selected to be included in the review. As a final step, 23 publications were selected as good representatives to be collated and summarized.

7.2. Methods and tools at the cross-company workshop

The literature review revealed how the methods and tools presented by the three companies at the cross-company workshop (Section 3) are reflected in the topics of recent academic research.

For example, Yan et al. (2024) propose an approach for root cause analysis based on a sanitizer, enhancing the accuracy of the analysis. Zivanovic et al. (2023) present an approach for applying combinatorial security testing to sanitizers. Borg et al. (2024) investigate the correlation between Code Health (measured by CodeScene) and levels of defect count. Kraus et al. (2024) propose an approach to identify input features for rarely visited code by performing sequential pattern mining for generator-based fuzzing. Beller et al. (2021) report experiences from using mutation testing at Facebook, and Petrovic et al. (2022) report experiences from Google.

7.3. Methods and tools at the focus groups

The participants at the focus groups (Section 4) recognized all the presented methods and tools from the cross-company workshop, but also added different tools for static code analysis to the same category of tools as CodeScene.

Static code analysis is also the topic of recent research publications, for example Schlichtig (2024) finds that static analysis tools are an important technique that helps in the development of secure code by analyzing code and reporting potential errors to developers, and analyses whether static analysis tools address the tools' usability sufficiently. Guan and Treude (2024) explore the integration of static analysis and additional context such as bug reports and design patterns into source code representations for deep learning models.

Venkatesh et al. (2024) state that the application of Large Language Models (LLMs) in static analysis tasks is a paradigm shift, and find that there is a need for specialized fine-tuning of LLMs to better suit specific static analysis tasks. Johansson et al. (2024) investigate the impact of compiler warnings on code quality in $C++$ projects, and find that compiler warnings correlate with lower code quality, with the clearest impact being on the number of critical issues in a project.

7.4. AI-based tools

It is notable that the participants at the workshops and the focus groups talked very little about other AI-based tools besides CodeScene. Recent published research, for example Fu et al. (2024), Yang et al. (2024) and Steenhoek et al. (2024), presents AI-based tools and techniques for fault localization and vulnerability detection. Recent publications such as Chen and Jabbarvand (2024), El Haji et al. (2024), Plein et al. (2024), and Zilberman and Betty Cheng (2024) also report investigations and experiences from using LLMs to generate or repair tests.

However, the reported experiences are not always positive: El Haji et al. (2024) find that within an existing test suite 54.72 % of generated tests are failing, broken, or empty tests, and without an existing test suite in place 92.45 % of the generated tests are failing, broken, or empty

tests. In a similar way, Zilberman and Betty Cheng (2024) find that LLMs frequently generate irrelevant tests that "suffer from numerous quality concerns". In response to this problem, Li and Shin (2024) propose a new LLM testing method called Mutation-based Consistency Testing.

7.5. Code review and code coverage

Code review was not mentioned by the participants at the validation workshop, although for example Mahbub et al. (2024) propose a semi-supervised learning-based system which assigns each code review a confidence score which is expected to resonate with the quality of the review.

The participants at the Company D focus group described how they questioned the value of code coverage. In recent research for example Dhulipala et al. (2024) present a tool which collaborates with LLMs to enhance code coverage prediction, and Ivanković et al. (2024) introduce "Productive Coverage" an approach to code coverage that guides developers to uncovered code that should be tested by (unit) tests. Interesting areas of further work could be to investigate whether adaptations of code coverage could recreate the developers' confidence in code coverage, and investigate how tools can support code reviews in various ways.

7.6. Testing AI generated code

The participants at the workshops and the focus groups did not discuss how testing of software units and components is affected by AI generated code, which can also be seen as something that makes testing of a software component an increasingly demanding task. However, there are voices that seem to argue that AI generated code is still suffering from severe quality problems, for example Siddiq et al. (2024) investigate the quality of ChatGPT generated code and found that generated code suffers from undefined and unused variables, and exception management-related issues.

The participants' interest in testing of software units and components with code primarily written by developers (and tested with tools and test cases written by developers) is also reflected in academic publications. For example, Neves et al. (2024) argue that in agile environments, developers need support from testing professionals to improve quality.

7.7. Summary and analysis

The literature review revealed how the *methods and tools presented by the three companies at the cross-company workshop and the focus groups are also reflected in the topics of recent academic research.*

The review also provided several examples of methods and tools not discussed by the participants at the workshops and the focus groups, that provide input to further work: AI-based tools for fault localization and vulnerability detection that might be a good complement to existing tools for static code analysis. Tools supporting code reviews could be a complement to static and dynamic testing. New methods and tools measuring code coverage (percentage measure of the degree to which the source code of a program is executed) and test coverage (percentage of software requirements that are tested) could complement mutation testing to improve test suite effectiveness.

8. Threats to validity

8.1. Threats to construct validity

It is plausible that a different set of questions and a different context for the workshops or the focus groups could have led to a different focus in the participants' responses. In order to handle these threats against construct validity, the pre-defined questions were purposely open-ended to open up to discussions among the participants. In addition to this, the results from this study (e.g. the four categories) should not be seen as complete, which is also discussed in Section 6.2. The participants at the workshops and the focus groups were selected as good informants with appropriate roles in the companies, following the guidelines from C. Robson and McCartan (2016).

Researcher bias could be a threat to reliability and construct validity during the interpretation of results from the cross-company workshop or the focus groups, especially as the study was conducted by a single researcher. This threat was mitigated with member checking (returning to participants and presenting the interpretations at the validation workshop), following the guidelines from C. Robson and McCartan (2016) who consider this to be "a very valuable means of guarding against researcher bias" and a good way to "amplify and understand the findings".

This publication also presents background material for both the participants and the studied companies in order to provide as much information as possible about the context and enable reproducibility of the study, as well as secondary studies based on the results from this study. The literature review was not conducted as a systematic literature review according to guidelines from e.g. Kitchenham (2004), but instead designed to provide good examples of related academic publications, answering the questions driving the review. Due to this, the literature review is difficult to reproduce. As the conclusions presented in Section 9 are based on the workshops and the focus groups (presented in Sections 3–5) and not on the results from the literature review (Section 7), this is not considered to be a threat to the reliability of the study.

8.2. Threats to internal validity

Of the 12 threats to internal validity listed by Cook, Campbell and Day (1979), Selection, Ambiguity about causal direction and Compensatory rivalry are relevant to this research study:

- *Selection*: All participants in the workshops and the focus groups in the study were purposely sampled in line with the guidelines for qualitative data appropriateness given by C. Robson and McCartan (2016). The selection of participants was informed by senior experts on software testing at each of the companies. Based on the rationale of these samplings and supported by Robson and McCartan, who consider this type of sampling superior for this type of study in order to secure appropriateness, this threat is considered to be mitigated.
- *Ambiguity about causal direction*: Statements that include cause and effect are collected from the workshops and the focus groups, and not introduced in the interpretation of the data. Therefore, as the conclusions are based on the data collected from the statements and discussions with the representatives from the six companies, this threat is considered to be mitigated.
- *Compensatory rivalry*: When holding focus groups or workshops, the threat of compensatory rivalry must always be considered. The questions for the workshops and the focus groups were deliberately designed to not judge performance or skills of the participants as individuals. Generally, the questions were also designed to be opened-ended to avoid any type of bias and ensure answers that were open and accurate. However, experiences from our previous work are that participants are more prone to self-criticism than to self-praise.

8.3. Threats to external validity

The list of conclusions presented in Section 9 was confirmed at the validation workshop with participants from the same companies as in the cross-company workshop and the three focus groups (internal generalizability). Due to this, it is conceivable that the findings from this study are only valid for these companies, or for companies that operate in the same industry segments and have similar characteristics (presented in Section 2.4).

Because of the diverse nature of the six companies, the companies

included in the study represent a good cross-section of large-scale companies the industry (as described in Section 2.4). Based on analytical generalization (Runeson and Höst 2009; Wieringa and Daneva 2015) it is reasonable to expect that results from the study are also relevant to a large segment of the software industry at large (external generalizability). However, one-person software startups are also part of the software industry, but their ways of working with methods and tools are most likely quite different. External validation in other companies (preferably in different industry segments and smaller organizations) is a natural suggestion for further work.

9. Conclusions and future work

9.1. Conclusions

This paper has presented insights from practitioners at six large-scale industrial companies, describing how they use different methods and tools to test software units and components in large-scale and complex software systems. The study included a cross-company workshop with 30 participants, three focus groups with a total of 19 participants, and a validation workshop with 30 participants representing the six studied companies.

The cross-company workshop (Section 3) revealed that *testing of a software component is an increasingly demanding task*, as our software systems are becoming increasingly larger and more complex. More components and more configurations, as well as more dependencies between software modules and data-driven behavior are cost-drivers for testing, calling for other methods and tools that can complement traditional scripted test cases. As every tool is also a cost for the organization, there is a need for a structured and holistic approach to how methods and tools should be used.

Practitioners in the software industry describe how there is a need to cut costs and save time, and to identify problems earlier. In response to this, the findings from this study provide three actionable strategies (Section 6) for how testing of units and components in the continuous integration and delivery pipeline can be conducted in a more structured way, which can be used to increase test efficiency:

1. *Evaluate and select methods and tools based on the four categories*, representing test activities with methods and tools for different purposes: Optimize feedback loops, Improve test efficiency, Improve test suite effectiveness, and Technical debt management.
2. *Finetune the feedback from the test activities in the pipeline* to ensure it is understandable (describing what the problem is), actionable (describing what the developer needs to do), and explained (describing and motivating why this is important).
3. *Provide the right support in ways-of-working and training programs* for all methods and tools used for testing software units and components. This support should be tailored for tools used by a few experts or by all developers, balancing support for the tool itself and how it is used in the organization.

The three actionable strategies are based on the analysis of the cross-company workshop (Section 3), the focus groups (Section 4) and the validation workshop (Section 5) with all six companies. The comparison with recent research (Section 7) revealed how the methods and tools presented by the three companies at the cross-company workshop and the focus groups are also reflected in the topics of recent academic research.

As the six case study companies included in this study operate in different industry segments, it is reasonable to expect that the results from the study can be applied to a large segment of the software industry. The results from the study can be valuable for both researchers and practitioners as it provides a systematic and holistic approach, rather than making changes blindly and hoping for the best.

9.2. Future work

The results from the study are based on workshops and focus groups where practitioners from industry discussed testing of software units and components, but it is possible that the results are also applicable to other types of testing. Future work could also expand the study to include other types of testing, for example to also include methods and tools used for testing functions and systems. An extended study could also include companies in other industry segments and smaller organizations, to further strengthen the generalizability of the results from the study.

At the validation workshop, participants from Company C and Company F in particular expressed different views on whether and how technical debt should be included in an automated pipeline, which proposes an idea for future work to investigate stakeholders for tools that measure technical debt, and propose solutions for when technical debt should be included in a continuous integration and delivery pipeline.

Previous work has investigated possible strategies for measuring and managing test diversity, to optimize test suites and automatically select or prioritize large sets of test cases (De Oliveira Neto 2018). Nikolik (2006) define test diversity as "a method for measuring the variety of software control flow and data flow". Feldt et al. (2008) present a model for test variability and propose the use of a theoretically optimal diversity metric at variation points in the model. As previous research has focused on test diversity with regards to scripted test cases, an interesting topic for further research could be to expand this to also include different types of tools and testing for different purposes.

CRediT authorship contribution statement

Torvald Mårtensson: Writing – original draft, Visualization, Validation, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Torvald Martensson reports financial support was provided by Saab AB. Torvald Martensson reports a relationship with Saab AB that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The author would like to thank all the participating engineers for their insights, patience and willingness to share their experiences and data.

Data availability

The data that has been used is confidential.

References

Beller, M., Wong, C.-P., Bader, J., Chandra, S., Meijer, E., 2021. What it would take to use mutation testing in industry - a study at facebook. In: Proceedings - International Conference on Software Engineering, pp. 268–277.
Birks, M., Mills, J., 2023. Grounded Theory: a Practical Guide, 3rd edition. SAGE.
Borg, M., Pruvost, I., Mones, E., Tornhill, A., 2024. Increasing, not diminishing: investigating the returns of highly maintainable code. In: Proceedings - 2024 ACM/IEEE International Conference on Technical Debt, TechDebt 2024, pp. 21–30.
Brown, J.B., 1999. The use of focus groups in clinical research. In: Crabtree, B.F., Miller, W.L. (Eds.), Doing Qualitative Research, 2nd ed. Sage, Thousand Oaks, CA, pp. 109–124.

Bryant, A., 2009. Grounded theory and pragmatism: the curious case of Anselm Strauss. Forum Qual. Sozialforsch. 10 (3).
Charmaz, K., 2025. Constructing Grounded Theory, 3rd edition. SAGE.
Chen, Y., Jabbarvand, R., 2024. Can ChatGPT repair non-order-dependent flaky tests?. In: Proceedings - 2024 IEEE/ACM International Flaky Tests Workshop, FTW 2024, pp. 22–29.
Cook, T.D., Campbell, D.T., Day, A., 1979. Quasi-Experimentation: Design & Analysis Issues for Field Settings, 351. Houghton Mifflin Boston.
Crispin, L., Gregory, J., 2009. Agile testing. Addison-Wesley.
Dhulipala, H., Yadavally, A., Nguyen, T.N., 2024. Planning to guide LLM for code coverage prediction. In: Proceedings - 2024 IEEE/ACM 1st International Conference on AI Foundation Models and Software Engineering, FORGE 2024, pp. 24–34.
Dobslaw, F., Wan, R., Hao, Y., 2023. Generic and industrial scale many-criteria regression test selection. J. Syst. Softw. 205, 111802.
Durelli, V.H.S., Durelli, R.S., Borges, S.S., Dias, D.R.C., Guimarães, M.P., 2019. Mchine learning applied to software testing: a systematic mapping study. IEEe Trans. Reliab. 68 (3), 1189–1212, 8638573.
Elbaum, S., Rothermel, G., Penix, J., 2014. Techniques for improving regression testing in continuous integration development environments. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2014, pp. 235–245.
El Haji, K., Brandt, C., Zaidman, A., 2024. Using GitHub copilot for test generation in Python: an empirical study. In: Proceedings - 2024 IEEE/ACM International Conference on Automation of Software Test, AST 2024, pp. 45–55.
Fern, E.F., 1982. Use of focus groups for idea generation: effects of group size, acquaintanceship and moderator on response quantity and quality. J. Market. Res. 19 (1), 1–13.
Feldt, R., Torkar, R., Gorschek, T., Afzal, W., 2008. Searching for cognitively diverse tests: towards universal test diversity metrics. In: 2008 IEEE International Conference on Software Testing Verification and Validation Workshop, ICSTW'08, pp. 178–186, 4567005.
Fu, M., Tantithamthavorn, C., Le, T., Phung, D., Grundy, J., 2024. AIBugHunter: a practical tool for predicting, classifying and repairing software vulnerabilities. Empir. Softw. Eng. 29 (1), 4.
De Oliveira Neto, F.G., Feldt, R., Erlenhov, L., Nunes, J.B.D.S., 2018. Visualizing test diversity to support test optimisation. In: Proceedings - Asia-Pacific Software Engineering Conference, APSEC, 2018-December, pp. 149–158, 8719537.
Greenbaum, T.L., 2003. Focus group research–why the traditional research methodology works so effectively and why it deserves to be the most respected of all qualitative research tools. Quirk's Market. Res. Rev. 1–5.
Guan, X., Treude, C., 2024. Enhancing source code representations for deep learning with static analysis. In: IEEE International Conference on Program Comprehension, pp. 64–68.
Hughes, D., DuMont, K., 1993. Using focus groups to facilitate culturally anchored research. Am. J. Community Psychol. 21 (6), 747–773.
ICSE, 2024. IEEE/ACM International Conference on Software Engineering 2024 with co-hosted conferences [online, accessed 30-October-2024]. https://conf.researchr.org/home/icse-2024 [online, accessed 30-October-2024].
ICST, 2024. IEEE International Conference on Software Testing, Verification and Validation 2024 with co-hosted conferences [online, accessed 30-October-2024]. https://conf.researchr.org/home/icst-2024 [online, accessed 30-October-2024].
ISTQB, 2024. The International Software Testing Qualifications Board Standard Glossary of Terms, V4.5 [online, accessed 5-November-2024]. https://www.istqb.org/ [online, accessed 5-November-2024].
Ivanković, M., Petrović, G., Kulizhskaya, Y., Just, R., Fraser, G., 2024. Productive coverage: improving the actionability of code coverage. In: ACM International Conference Proceeding Series, pp. 58–68.
Johansson, A., Holmberg, C., Neto, F.G.D.O., Leitner, P., 2024. The impact of compiler warnings on code quality in C++ projects. In: IEEE International Conference on Program Comprehension, pp. 270–279.
Kitchenham, B., 2004. Procedures for performing systematic reviews. Keele UK Keele Univ. 33 (2004), 1–26.
Kitzinger, J., 1995. Introducing focus groups. Br. Med. J. 311, 299–302.
Kraus, R., Nguyen, H.L., Schneider, M.A., 2024. Generator-based fuzzing with input features. In: Proceedings - 2024 IEEE/ACM International Workshop on Search-Based and Fuzz Testing, SBFT 2024, pp. 13–20.
Li, Z., Shin, D., 2024. Mutation-based consistency testing for evaluating the code understanding capability of llms. In: Proceedings - 2024 IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI, CAIN 2024, pp. 150–159.
Mahbub, S., Arafat, M.E., Rahman, C.R., Ferdows, Z., Hasan, M., 2024. ReviewRanker: a semi-supervised learning based approach for code review quality estimation. In: Proceedings - International Conference on Software Engineering, pp. 362–363.
Meszaros, G., 2007. XUnit Test Patterns: Refactoring Test Code. Addisson Wesley.
Myers, G.J., Sandler, C., Badgett, T., 2011. The Art of Software Testing, 3rd Ed. Wiley.
Mårtensson, T., Ståhl, D., Bosch, J., 2019. Test activities in the continuous integration and delivery pipeline. J. Softw.: Evolut. Process 31 (4).
Mårtensson, T., Ancher, G., Ståhl, D., 2023. Test environments for large-scale software systems—An industrial study of intrinsic and extrinsic success factors. Softw. Test. Verif. Reliab. 33 (3), e1839.
Neves, L., Campos, O., Santos, R., Santos, I., Santos, R.D.S., 2024. Elevating software quality in agile environments: the role of testing professionals in unit testing. In: Proceedings - 2024 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2024, pp. 293–296.
Nikolik, B., Test diversity, information and Software technology, 48(11), pp. 1083–1094 (2006).
Orso, A., Shi, N., Harrold, M.J., 2004. Scaling regression testing to large software systems. In: Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering, pp. 241–251.
Petrovic, G., Ivankovic, M., Fraser, G., Just, R., 2022. Practical mutation testing at scale: a view from Google. IEEE Trans. Softw. Eng. 48 (10), 3900–3912.
Plein, L., Ouédraogo, W.C., Klein, J., Bissyandé, T.F., 2024. Automatic generation of test cases based on bug reports: a feasibility study with large language models. In: Proceedings - International Conference on Software Engineering, pp. 360–361.
Powell, R.A., Single, H.M., 1996. Focus groups. Int. J. Qual. Health Care 8 (5), 499–504.
Powers, D., 2011. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness & correlation. J. Mach. Learn. Technol. 2, 37–63.
Robson, C., McCartan, K., 2016a. Real World Research, 4th ed. Wiley.
Rodríguez, P., 2024. Grounded theory in software engineering: challenges and lessons learned from the trenches. In: Proceedings - 2024 IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering, WSESE 2024, pp. 21–26.
Robson, C., McCartan, K., 2016b. Real World Research, 4th ed. Wiley.
Runeson, P., Höst, M., 2009. Guidelines for conducting and reporting case study research in software engineering. Empir. Softw. Eng. 14 (2), 131–164.
Shamsuddin, A., Sheikh, A., Keers, R.N., 2021. Conducting research using online workshops during COVID-19: lessons for and beyond the pandemic. Int. J. Qual. Methods 20.
Schlichtig, M., 2024. Building a framework to improve the user experience of static analysis tools. In: Proceedings - International Conference on Software Engineering, pp. 165–169.
Siddiq, M.L., Roney, L., Zhang, J., Santos, J.C.S., 2024. Quality assessment of ChatGPT generated code and their use by developers. In: Proceedings - 2024 IEEE/ACM 21st International Conference on Mining Software Repositories, MSR 2024, pp. 152–156.
Steenhoek, B., Gao, H., Le, W., 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In: Proceedings - International Conference on Software Engineering.
Ståhl, D., Mårtensson, T., 2021. Won't somebody please think of the tests? A grounded theory approach to industry challenges in continuous practices. In: Proceedings - 2021 47th Euromicro Conference on Software Engineering and Advanced Applications, SEAA 2021, pp. 70–77.
Thornberg, R., Charmaz, K., 2011. Grounded theory. In: Lapan, S., Quartaroli, M., Riemer, F. (Eds.), Qualitative Research: An Introduction to Methods and Designs. Jossey-Bass, pp. 41–67.
Trudova, A., Dolezel. M., Buchalcevova, A., Artificial intelligence in software test automation: a systematic literature review. 181–192 (2020).
Venkatesh, A.P.S., Sabu, S., Mir, A.M., Reis, S., Bodden, E., 2024. The emergence of large language models in static analysis: a first look through micro-benchmarks. In: Proceedings - 2024 IEEE/ACM 1st International Conference on AI Foundation Models and Software Engineering, FORGE 2024, pp. 35–39.
Wieringa, R., Daneva, M., 2015. Six strategies for generalizing software engineering theories. Sci. Comput. Program. 101, 136–152.
Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: 18th International Conference on Evaluation and Assessment in Software Engineering. EASE.
Yan, Z., Haipeng, Q., Ying, L., Liu, K., Chao, Q., 2024. Enhanced fast and reliable statistical vulnerability root cause analysis with sanitizer. In: Proceedings - 2024 IEEE Conference on Software Testing, Verification and Validation, ICST 2024, pp. 47–58.
Yang, A.Z.H., Martins, R., Le Goues, C., Hellendoorn, V.J., 2024. Large language models for test-free fault localization. In: Proceedings - International Conference on Software Engineering.
Yoo, S., Harman, M., 2012. Regression testing minimization, selection and prioritization: a survey. Softw. Test., Verificat. Reliab. 22 (2), 67–120.
Zivanovic, J., Leithner, M., Simos, D.E., Pitzer, M., Slanina, P.J., 2023. Combinatorial methods for HTML sanitizer security testing. In: Proceedings - 2023 IEEE 16th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2023, pp. 255–259.
Zilberman, S., Betty Cheng, H.C., 2024. No free lunch' when using large language models to verify self-generated programs. In: Proceedings - 2024 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2024, pp. 29–36.
Ørngreen, R., Levinsen, K., 2017. Workshops as a research methodology. Electron. J. e-Learn. 15 (1), 70–81.

Torvald Mårtensson is Technical Fellow within System Integration Testing at Saab AB. He has a background of twenty years in the aeronautics industry and another eight years in the telecom industry. His work has primarily revolved around systems integration and system testing of large-scale and complex software systems. Torvald is also Associate Professor (Senior Lecturer) at Linköping University. He received an MSc degree from Linköping University, Sweden, and a PhD degree from the University of Groningen, The Netherlands.